Realising the skip-count trick

Contents

1 Introduction 1

2 Notation 2

3 Additional substructure below the shorter path 2

4 Rule 2r 3

5 Rule 2i 4

6 Similarities with rule 3 active data updates 6

7 Conclusion 6

1 Introduction

In this document I will describe where the skip-count trick is used and how to construct strings that require it. It is difficult to explain with an example during the tutorials as the required strings can be quite long and it can be tedious to construct such suffix trees by hand. I hope that by the end of this document you will understand that lengthy strings are not necessary.

This will be a long document and is **not** required reading. It should prove to be useful if you implement Ukkonen's algorithm. Also, the discussion on how to construct a string that realises the skip-count trick should be useful for understanding how to construct interesting test cases. After reading this post I hope you will have gained:

- Further insight into suffix links, how to handle their traversal, and the skip-count rule.
- The ability to construct test cases that probe the skip-count rule.
- A few explicit examples of interesting test cases. This will be very useful if you implement Ukkonen's algorithm.

Throughout this document I make a few claims and ask you to consider why/if these are true. These questions are labelled "**QUESTION** <**N**>". Feel free to answer these questions on the associated forum thread. Please also ask if you have other questions.

Prepared by Lachlan Morris

2 Notation

Let us introduce the following notation:

- Lower-case letters represent a single character, e.g. `c` is the letter `'c'`. Upper-case letters represent a string, e.g. `A` could represent `'abc'` or just `'c'`.
- Straight lines represent an edge. Squiggly lines represent sub-paths, along which there could be internal nodes. A straight line going from node N_1 to node N_2 means there are no nodes between N_1 and N_2, whereas a squiggly line between nodes N_3 and N_4 means there could be nodes between N_3 and N_4.
- Vertical pipes surrounding a string represents the length of the string. E.g. $|$`'abc'`$| = 3$.

First, we will identify what structure we need within the suffix tree to realise a case where the skip-count trick can be used. The skip-count trick is used after a rule 2. There are two cases to consider:

- Rule 2r: performing a rule 2 directly below the root node. The active/parent node is the root node.
- Rule 2i: performing a rule 2 below an internal node that is not the root node. The active/parent node is an internal node.

Note: these two cases do not correspond to the two cases described in the lecture; I split them differently here because of how it affects traversal for the next extension.

3 Additional substructure below the shorter path

After a rule 2, we want to follow the active/parent node's suffix link and insert the next longest suffix. That is, after inserting `str[j..i]`, we follow the suffix link and insert `str[j+1..i]`. The skip-count trick is used to update the active data correctly so we can easily insert `str[j+1..i]`. The skip-count trick is required if the path we link to has additional substructure, as shown in Figure 1.

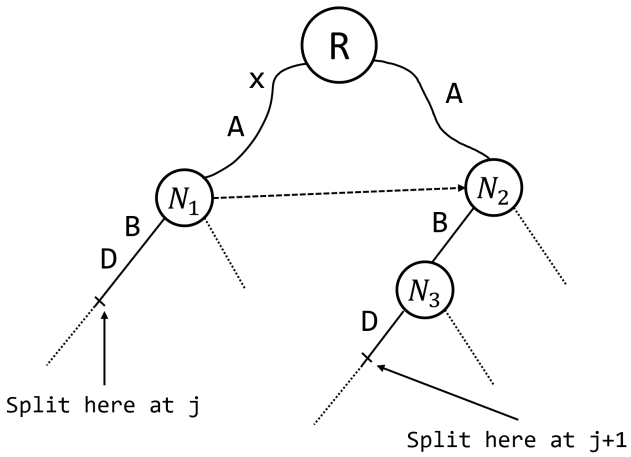

Figure 1: Abstract skip-count example with an active node (N_1) below the root.

In extension j, the active node is N_1, and we split the `B[1]` edge (remembering that `B` means a string, not just one character). When we follow N_1's suffix link to N_2 to prepare the active data for extension $j+1$, we find that the `B[1]` path below N_2 has more structure than the `B[1]` edge from N_1. This can happen, and we will construct strings to show this explicitly. Note that this can never happen the other way around. N_1 cannot have more structure below it than N_2 does – provided there is a suffix link going from N_1 and N_2. **QUESTION 1**: why is this true?

4 Rule 2r

We can see that Figure 1 corresponds to a rule 2i. We will return to this case once we have considered rule 2r, for which we can use a shorter example string. Figure 2 shows a rule 2r. This time the active node is the root node. When we insert just after `B` along the `'x'` edge in extension j, we follow the root node's suffix link to itself to prepare for extension $j+1$. We pick the `A[1]` edge. There is an internal node along the path `AB...` that wasn't present along the edge `xAB...`. This is due to the string having a substring `AD`, while not having a substring `xAD`. In extension j the active node was the root, and the remainder was `xAB`. After following the suffix link we reduce the remainder to `AB` because it was from the root to the root. However, our remainder `AB` takes us beyond the length of the `A[1]` edge from the root. We should instead skip down to the internal node N_1, set it as the active node, and set our remainder to `B`. The skip-count trick lets us skip down to N_1 in $\mathcal{O}(1)$ time, rather than $\mathcal{O}(|\texttt{A}|)$ time.

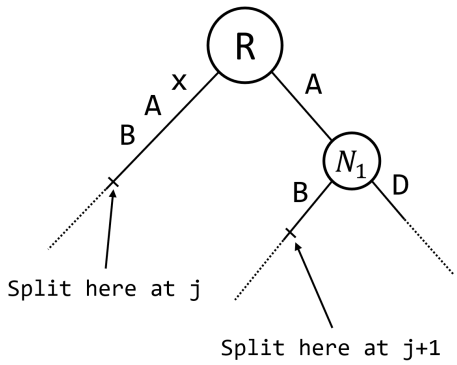

Figure 2: Abstract skip-count example with root as the active node.

Now that we have identified the structure within the suffix tree that allows for the skip-count trick, let us figure out what strings yield this structure. **Hint**: this might be a good test case if you want to check that your skip-count trick works!

For rule 2r, which is shorter, we do not need parent internal nodes. In Figure 2, we see that we need the following:

- A substring `xAB`.
- A substring `AD`.
- No substring `xAD`. (If `xAD` existed, where `B` $\neq$ `D`, we would have an internal node above the j split.)
- Two substrings `xABa` and `xABb`, where `'a'` $\neq$ `'b'`. (This causes the splitting at j.)

The shortest example is the string `xABADxABb`, where `A[1]` $\neq$ `'b'`. An explicit string would be `'xabadxabb'`.This simplest string already has 9 characters. **QUESTION 2**: can you think of a shorter string that realises the skip-count trick with case 2r?

5 Rule 2i

Now we will consider rule 2i. This time we need parent internal nodes. In Figure 1 (or Figure 3), we see that we need the following:

- A substring `xABD`.
- A substring `xAC`, where `B[1]` $\neq$ `C[1]`. (This gives the internal nodes N_1 and N_2.)
- A substring `ABE`, where `E[1]` $\neq$ `D[1]`. (This gives the internal node N_3.)
- No substring `xABE`. (Otherwise there would be an internal node between `B` and `D` below N_1.)
- Two substrings `xABDa` and `xABDb`, where `'a'` $\neq$ `'b'`. (This causes the splitting at j.)

The shortest example is the string `xABDxACABExABDy`, where `'x'` $\neq$ `'y'`. An explicit string would be `'xabdxacabexabdy'`. The simplest string has 15 characters. **QUESTION 3**: can you think of a shorter string that realises the skip-count trick with case 2i?

Let us look at the insertion `xABDy` in this abstract example `xABDxACABExABDy`. The current relevant parts of the suffix tree are shown in Figure 3. The active node is N_1 and the remainder is `BD`. We have a mismatch after the remainder, `'x'` $\neq$ `'y'`, so it is a rule 2a. We split the edge, by creating a new leaf for the suffix starting at j, and attaching it to a new internal node (N_4) located just after `xABD`. This is shown in Figure 4.

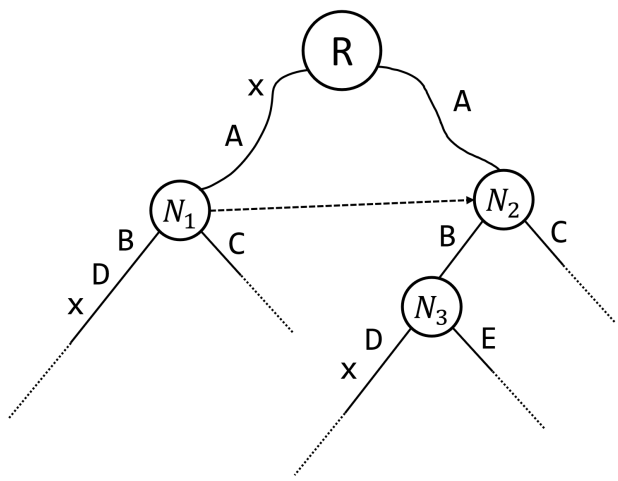

Figure 3: Setup for skip-count example, rule 2i.

We then follow N_1's suffix link to N_2, in preparation for the insertion of `ABDy` into the tree. The remainder is still `BD`. We do not reduce it because we have not followed the suffix link from the root to itself. **QUESTION 4**: why do we reduce the remainder if and only if the suffix link takes us from the root to the root? Using the skip-count trick, we pick the `B[1]` edge from N_2 and immediately skip to N_3 since $|\texttt{remainder}| \geq |\texttt{B}|$. We remove `B` from the remainder, leaving

`remainder` = `D`. Then we check if the next character after the remainder, `'x'`, matches `'y'`. It does not, so we do another rule 2a. Figure 5 shows the state of the tree after this extension.

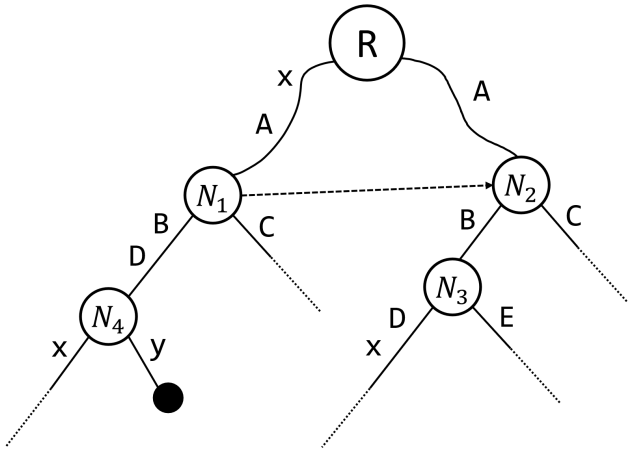

Figure 4: Skip-count example for rule 2i, with extension j performed.

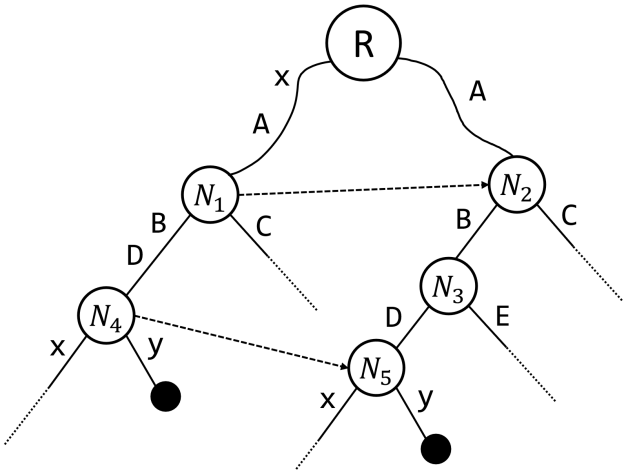

Figure 5: Skip-count example for rule 2i, with extension $j + 1$ performed.

We keep going from there, until the substring `'y'` has been inserted. Note this chain of rule 2's can be stopped by a rule 3. That is, at some point, we might find a `'y'` after `remainder`. **QUESTION 5**: can you think of a string that exhibits this property (i.e. a rule 3 occurs after skip-counting)?

An obvious final question along these lines is **QUESTION 6**: what string structure would exhibit a case where you skip multiple nodes during the skip-count trick in a single extension? And, one last example for you to consider is the string `'ababcbababd'`. This string has the advantage of having a small alphabet and overall length, while yielding a skip-count case. This means it should produce a simpler suffix tree to work with. Try to determine which skip-count case this example yields in particular, and how it differs from the other examples presented here.

6 Similarities with rule 3 active data updates

An important thing to remember is that rule 2's aren't the only times when you might need to update the active data (node, remainder, edge, etc.). If a series of rule 3's build a remainder that exceeds the active edge length, then the active node needs to be set to the internal child node. For instance, take the suffix tree given in Figure 3 and set `A` = `'a'` for simplicity. Consider a case where the active node is the root, the active edge is `'xa'`, the remainder is `'x'`, and we now want to insert `'xa'`. The `a`'s match which is a rule 3. The remainder increases to `'xa'` which is the entire current (active) edge. In this case, we should update the active node to N_1, set the remainder to the empty string, and we could set our active edge to the empty string or `None` (however you want to implement it – in fact you might have a different set of active data properties).

The way we handle the case where a rule 3 takes us to the next internal node has many similarities with what we do in the skip-count trick. This can be viewed as another case of the skip-count trick, but really it is just making sure the active data is updated correctly after an extension. The skip-count trick, on the other hand, is an optimisation to *skip* edges rapidly.

7 Conclusion

Hopefully this post gives you more insight into an important aspect of tree traversal, and even more importantly, a better understanding of how to construct interesting test cases. A good way to break down test case construction is:

- Choose a rule or trick to test.
- Determine what structure in the suffix tree is required.
- Construct a string that gives that structure.

Please post on the associated forum thread (or send an email) if you have any questions, answers, or spot any mistakes. Happy coding!